Kubernetes Alert Instructions for Grafana Alerts

This document maps Grafana alerts to Prometheus queries used on the Grafana MCP server to validate issues and identify affected Kubernetes resources.

🔔 Alert Name: Pods Not Ready Alert

Description:
Triggered when one or more pods are not in the "Running" phase.

Evaluation Steps:

- Identify pods not in the Running state.

Prometheus Query to list pods not running:

```
(kube_pod_status_ready{condition="true", cluster != "jacksonhole"} == 0)
```

Note: Use Grafana to filter pod and namespace labels for affected pods.

Recommendation:
Investigate pod logs and events for root cause (e.g., image pull errors, crash loops).

🔔 Alert Name: Namespace without Pods Alert

Description:
Fires when namespaces have zero running pods.

Evaluation Steps:

- Identify namespaces with no pods running.

Prometheus Query to list namespaces and their pod counts:

```
kube_namespace_created{namespace!~"kube-.*", cluster != "jacksonhole"}

 unless

kube_pod_created{cluster != "jacksonhole"}
```

To find namespaces with zero pods, compare this list with all namespaces.

Recommendation:
Review namespaces with no pods, remove unused namespaces or investigate why workloads are missing.

If a namespace is stuck in a Terminating state due to finalizers, patch the namespace by setting its metadata.finalizers field to null. This will remove all finalizers and allow Kubernetes to complete the deletion process.

🔔 Alert Name: CPU Utilization Alert

Description:
Triggered when CPU usage exceeds limits or requests.

Evaluation Steps:

- Identify pods with high CPU usage.
- Check CPU usage versus limits.

Prometheus Queries:

- Current CPU usage per pod:

rate(container_cpu_usage_seconds_total{image!="",container!="POD"}[5m])

Describe the pod to check the limit and request details

Recommendation:
Adjust CPU requests and limits as needed after approval.

🔔 Alert Name: Memory Utilization Alert

Description:
Triggered when memory usage exceeds limits or requests.

Evaluation Steps:

- Identify pods with high memory usage.
- Check memory usage versus limits.

Prometheus Queries:

- Current memory usage per pod:

container_memory_usage_bytes{image!="",container!="POD"}

Describe the pod to check the limit and request details

Recommendation:
Adjust memory requests and limits as needed after approval.

🔔 Alert Name: Inactive Namespaces Alert

Description:
Triggered when a namespace is stuck in the "Terminating" state.

Evaluation Steps:

- Identify namespaces in "Terminating" status.

Prometheus Query:
(kube_namespace_status_phase{phase="Active"} == 0)

Recommendation

If a namespace is stuck in a Terminating state due to finalizers, patch the namespace by setting its metadata.finalizers field to null. This will remove all finalizers and allow Kubernetes to complete the deletion process.

🔔 Alert Name: Unbound Persistent Volume Alert

Description:
Triggered when Persistent Volume Claims (PVCs) are unbound.

Evaluation Steps:

- Identify PVCs in unbound state.

Prometheus Query:
(kube_persistentvolumeclaim_status_phase{phase="Bound"} == 0)

🔔 Alert Name: Pods without Memory Limits Alert

Description:
Triggered when pods do not have memory limits set.

Evaluation Steps:

- Identify pods without memory limits.

Prometheus Queries:

- **List pods without memory limits:**

```
sum by(pod, namespace) (container_spec_memory_limit_bytes{cluster!="jacksonhole"}) == 0
```

Recommendation:
Recommend setting memory limits on these pods to avoid scheduling and eviction issues. To fix, get specific pod from the user and use current usage metric container_memory_usage_bytes to suggest limits by adding ~30% buffer.

To get the current memory usage, use the following query container_memory_usage_bytes{image!="",container!="POD"} with pod name

---

🔔 Alert Name: Pods without CPU Limits Alert

Description:
Triggered when pods do not have CPU limits set.

Evaluation Steps:

- Identify pods without CPU limits.

Prometheus Queries:

- **List pods without CPU limits:**

```
kube_pod_container_info unless on(container, pod, namespace, cluster)
kube_pod_container_resource_limits{resource="cpu"}
```

Recommendation:
Recommend setting CPU limits to avoid resource contention.
To fix, get specific pod from the user and use current usage metric container_cpu_usage_seconds_total to suggest limits by adding ~30% buffer.